A STUDY COMPANION

NumPy Book

Course Notes & Summaries

Notes & summaries from the **NumPy** video course by **JoeTech**

COMPILED BY YOUSSEF ✦ 2026

ABOUT THIS BOOK

NumPy Book is a personal study companion compiled while following the NumPy course by JoeTech on YouTube.

Each chapter distills one lesson from the series into readable notes, worked examples, and quick-reference tables.

Compiled by Youssef · first compiled August 2026

Not an official publication of JoeTech or the NumPy project.

Contents

CHAPTER 01

The ndarray — NumPy's Core Object

Sample chapter — JoeTech, NumPy Course, Video 1 (placeholder for design review)

NumPy's entire library is built around a single object: the **ndarray**, a fast, fixed-type, multi-dimensional array. Unlike a plain Python list, every element in an ndarray shares the same DTYPE, which is what lets NumPy store data compactly and operate on it at C speed instead of looping in pure Python.

Creating arrays

The most common entry point is `np.array()`, which converts a Python list (or list of lists) into an ndarray:

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([[1, 2, 3], [4, 5, 6]])

print(a.shape)   # (3,)
print(b.shape)   # (2, 3)
print(b.dtype)   # int64
```

NumPy also ships several convenience constructors for arrays with a known pattern, so you rarely need to build one element at a time:

Function	Produces
`np.zeros((r, c))`	array filled with `0.0`

Function	Produces
`np.ones((r, c))`	array filled with `1.0`
`np.arange(start, stop, step)`	evenly spaced values, like `range()`
`np.linspace(start, stop, n)`	`n` evenly spaced values, inclusive of both ends
`np.eye(n)`	an `n x n` identity matrix

> NOTE
>
> A NumPy array's `dtype` is fixed at creation. Assigning a float into an integer array silently truncates it instead of raising an error — this is one of the most common sources of subtle bugs for people coming from plain Python lists.

Shape, axes, and reshaping

Every ndarray carries a `.shape` tuple describing its size along each axis, and a `.ndim` giving the number of axes. Reshaping does not copy data — it changes how the same underlying buffer is *viewed*:

```
c = np.arange(12)
grid = c.reshape(3, 4)

print(grid)
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]]

print(grid.T)      # transpose: swap the two axes
print(grid.flatten())  # back to 1-D, as a copy
```

Because `reshape` returns a *view* whenever possible, modifying `grid` after this point can also change `c` — the two arrays point at the same memory. This is the same theme that comes up constantly in NumPy: operations are fast precisely because they avoid copying, so it pays to know which ones share memory and which ones don't.

Indexing and slicing

Indexing an ndarray looks like indexing a list, but extends naturally to multiple dimensions using a comma inside the brackets:

```
grid[0, 1]      # element at row 0, column 1 → 1
grid[:, 0]      # every row, column 0        → [0, 4, 8]
grid[1:, :2]    # rows 1 onward, first two columns
```

> **TIP**
>
> A slice of an ndarray is a *view*, not a copy. If you need an independent array, call `.copy()` explicitly — e.g. `sub = grid[1:, :2].copy()`.

Why this matters

Everything covered in the rest of the course — broadcasting, vectorised math, aggregation along an axis — builds directly on these three ideas: a fixed dtype, a shape you can reinterpret without copying, and indexing that generalises cleanly to N dimensions. Getting comfortable with `.shape`, `.dtype`, and the view-vs-copy distinction now makes every later chapter easier.